

SPLASH: SPeculative Leakage-Adaptive Secure Hardware

Huimin Li¹, Mohamadreza Rostami², Pallavi Borkar³, Lichao Wu⁴, and
Ahmad-Reza Sadeghi²

¹ Shenzhen University of Advanced Technology, Shenzhen, China,
`lihuimin@suat-sz.edu.cn`

² Technical University of Darmstadt, Darmstadt, Germany,
`mohamadreza.rostami@trust.tu-darmstadt.de`,
`ahmad.sadeghi@trust.informatik.tu-darmstadt.de`

³ Indian Institute of Technology Madras, Chennai, India,
`cs20d202@smail.iitm.ac.in`

⁴ University of Bristol, Bristol, United Kingdom, `lichao.wu@bristol.ac.uk`

Abstract. Modern processors are largely fixed at the time of fabrication, rendering post-silicon security updates infeasible. This lack of flexibility is especially problematic for speculative execution attacks, which exploit microarchitectural optimizations to leak sensitive information through transient execution. However, existing defenses are typically hardwired, narrowly scoped, and non-adaptive. Once deployed, these defenses lack the flexibility to respond to new attack variants, creating a critical security gap.

This paper presents SPLASH, a novel and adaptive framework for comprehensively mitigating speculative execution vulnerabilities across processor components. SPLASH introduces two primary contributions: (i) we introduce *Speculative Information Flow Tracking* (SIFT), which enables fine-grained tracking of speculative data propagation throughout the microarchitecture; (ii) the *reconfigurable speculative table* provides, for the first time, enforceable and fully configurable control over speculative behavior across the processor pipeline. SPLASH is runtime-configurable, allowing dynamic security policy adjustments post-fabrication, such as tuning the speculative window size or selectively protecting specific microarchitectural structures, without requiring hardware redesign.

We implement SPLASH on both the small and medium BOOM processors and evaluate its effectiveness in mitigating speculative leakage vulnerabilities. SPLASH successfully mitigates all types of speculative execution attacks. In terms of computation overhead, SPLASH incurs only 0.05% overhead on small BOOM and 1.23% on medium BOOM on average compared with the baseline. Compared to the best performing state-of-the-art defenses, SPLASH reduces overhead by 129.2× and 1.2×, respectively, with negligible hardware cost.

Keywords: Speculative Leakage, Reconfigurable Hardware, Microarchitectural Security

1 Introduction

Hardware security is the cornerstone of the confidentiality and integrity of modern computing systems. Vulnerabilities in hardware allow attackers to exploit design flaws or misuse inherent functionalities, exposing sensitive data and compromising overall system security. A prominent class of such vulnerabilities stems from transient execution, temporary instruction sequences executed speculatively or out-of-order by modern CPUs to optimize performance [11]. Although the results of these executions are eventually discarded and do not alter the architectural state, they can leave observable traces in microarchitectural components such as caches or branch predictors. Transient execution attacks have been shown to breach critical isolation boundaries, leaking sensitive data such as encryption keys, passwords, user data, and kernel memory contents across privilege levels and between processes [38, 43, 44]. Various hardware modules become the attack target, including caches [22, 27, 43], buffers [10, 30, 38], and the register files [31]. The evolution of these attacks highlights the emergence of new threats in the hardware system.

Speculative execution attacks, a critical subset of transient execution attacks, have received significant attention due to their practicality and impact [11, 22]. These attacks exploit speculative control flow paths to leak sensitive information via microarchitectural side channels, even when the speculative instructions are eventually discarded [22]. Mitigating such attacks presents significant challenges, particularly in post-silicon systems where hardware modifications are no longer feasible. Software-based protections, such as inserting speculative execution barriers (e.g., **FENCE** instructions), can be deployed but introduce significant performance overhead [49, 50]. Microcode updates may address a limited subset of vulnerabilities but are inadequate for comprehensive attacks [18]. In contrast, pre-silicon hardware-based mitigations offer long-term effectiveness as they address speculative execution vulnerabilities directly at the microarchitectural level. Unfortunately, existing hardware-based countermeasures are often rigid and narrowly focused, designed to address only specific known speculative execution attacks and to protect particular CPU components. For instance, defenses such as Conditional Speculation [26] and SpecLFB [12] target only the Data Cache (D-Cache), leaving other parts of the system vulnerable to evolving or unforeseen speculative execution threats. Moreover, although being hardware-based solutions, many introduce significant performance or size penalties. Solutions like storing speculatively loaded data in a shadow buffer [20] or delaying speculative loads until they reach the head of the reorder buffer (ROB) [37], which potentially incurs performance overheads exceeding 20% [36, 50]. As a result, these approaches often introduce significant complexity and inefficiency into hardware designs. These limitations underscore the need for more flexible, scalable, and lightweight protection against both current and emerging speculative execution attacks.

Our Goal and Contributions. This paper presents **SPLASH**, a novel framework to mitigate speculative execution vulnerabilities without imposing con-

straints on specific hardware modules. SPLASH adopts a hybrid enforcement–control architecture, in which security enforcement is implemented entirely in hardware, while protection policies and coverage are configured by privileged software at runtime. Specifically, SPLASH relies on hardwired microarchitectural mechanisms to track speculative information flow and precisely restore the microarchitectural state upon mis-speculation. In contrast, software interacts with SPLASH solely through Control and Status Registers (CSRs) to configure protection parameters, such as selecting protected microarchitectural components, bounding the speculative window, and specifying security-critical regions. This separation between hardware-enforced security mechanisms and software-configured policies enables SPLASH to achieve both high efficiency and post-deployment adaptability. SPLASH incurs minimal performance overhead under low-risk execution scenarios, while allowing more comprehensive protection to be dynamically activated when higher security assurance is required. By avoiding software instrumentation and preserving a purely hardware-based enforcement path, SPLASH provides a practical, scalable, and future-proof solution for defending modern processors against evolving speculative execution attacks. Our contributions are:

- We introduce **SPLASH**, the first reconfigurable framework that delivers comprehensive, cross-component mitigation of speculative execution attacks, generalizing protection across different components without sacrificing performance scalability.
- We design and implement *Speculative Information Flow Tracking (SIFT)*, a novel hardware mechanism that precisely tracks speculative data propagation in-flight, offering fine-grained control with minimal computational and performance overhead.
- We propose a *runtime-bounded speculative window mechanism* that dynamically customizes the ROB size for untrusted code segments. This mechanism balances rollback overhead with attack surface reduction, providing adaptive mitigation that does not compromise execution throughput.
- We develop a *selective enforcement model* based on configurable instruction memory regions, enabling SPLASH to enforce protections only within security-critical code, thereby avoiding unnecessary performance penalties on benign or non-sensitive execution paths.
- We implement a programmable *Speculative Table (Spec-Table)*, a compact, hardware-supported structure configured via CSRs. The Spec-Table enables workload-aware, threat-adaptive policies at runtime, enhancing SPLASH’s flexibility across diverse application scenarios. By bounding the speculative window at runtime, SPLASH enables stronger protection for security-critical code while maintaining near-baseline performance for benign workloads, avoiding the worst-case slowdowns and rigidity of existing hardware-based defenses.
- We implement SPLASH in the open-source RISC-V BOOM processor and perform an extensive evaluation using standard benchmarks and representative Spectre-class attacks. Our results show that SPLASH prevents all speculative execution attacks that leave residual microarchitectural state in

BOOM, while incurring near-zero performance overhead (only 0.05% overhead on small BOOM and 1.23% on medium BOOM relative to baseline execution on average) and modest hardware cost (on average 6.65% more LUTs and 0.70% more flip-flops than the baseline design), with negligible overhead when extended beyond the data cache.

2 Background

2.1 Speculative Execution

Modern processors exploit instruction-level parallelism (ILP) through techniques such as out-of-order and speculative execution, allowing multiple instructions to execute concurrently once their dependencies are resolved [15, 40, 45]. Although instructions may execute out of program order, components like ROB ensure they retire in order, thereby preserving the correct architectural state.

The *speculative window*, the interval between the initiation of speculation and its resolution, is primarily constrained by the ROB’s capacity. Its effective size is typically approximated by the distance between the ROB’s head and tail pointers, representing the number of in-flight instructions, including speculative ones [28, 39, 49]. A wider speculative window allows for deeper speculation and greater parallelism, potentially improving performance; conversely, a narrower window limits speculative depth and can restrict performance gains.

To enable speculation, branch predictors [25] and memory disambiguators [22] predict control and data dependencies, respectively, allowing execution to proceed along speculative paths. When predictions are correct, instructions at the ROB head commit successfully. In cases of misprediction, speculative instructions are squashed, and the pipeline is flushed to a correct execution state. Despite the above mechanism, mis-speculated speculative instructions often leave persistent state changes in shared microarchitectural components, such as caches and TLBs, which can be exploited as side channels. In this work, we focus on speculative execution attacks that leverage invalid or transient microarchitectural states to leak sensitive information.

2.2 Speculative Execution Attacks

Speculative execution is a fundamental performance optimization employed in modern CPUs to maximize instruction-level parallelism. By predicting the outcomes of branches and resolving memory dependencies, processors can execute instructions ahead of time, rather than stalling while waiting for unresolved conditions. However, this aggressive speculation introduces significant security risks, as instructions executed along mispredicted paths, though eventually discarded architecturally, can leave microarchitectural side effects. These residual traces in hardware structures such as caches, branch predictors, TLB, and execution units form the foundation of *speculative leakages* [22, 30, 31]. Speculative execution attacks exploit these leakages.

In speculative execution attacks, the adversary manipulates the processor’s control flow or memory dependencies to deliberately trigger speculation along sensitive code paths. Although speculative instructions are ultimately squashed, their transient execution influences shared hardware resources in observable ways. By measuring changes in microarchitectural state, such as cache access latencies or branch predictor states, attackers can infer sensitive information, including cryptographic keys, user data, and kernel memory [22, 30, 31].

Multiple classes of speculative execution attacks have been demonstrated, each targeting different speculative mechanisms within modern CPUs. Bounds Check Bypass (BCB) [22] exploits conditional branch prediction to bypass array bounds checks, enabling speculative out-of-bounds memory accesses. Branch Target Injection (BTI) [22] targets indirect branch prediction by poisoning branch target buffers (BTBs), redirecting speculative execution toward attacker-controlled code sequences known as gadgets. Speculative Store Bypass (SSB) [2] exploits vulnerabilities in store-to-load forwarding, enabling speculative loads to bypass prior stores and potentially expose stale or unauthorized data. Finally, SpectreRSB [23] trains the return stack buffer (RSB) to mispredict return addresses, steering speculative execution along attacker-specified paths. To better illustrate the attack, consider the Bounds Check Bypass (Spectre Variant 1) attack, which leverages mistrained conditional branch predictors to subvert standard array bounds checks. An attacker controls the index variable `i` and trains the branch predictor to expect in-bounds conditions. Then, during speculative execution, a carefully crafted out-of-bounds index transiently accesses sensitive data, leaving detectable cache traces, as shown in Listing 1:

Listing 1 Example of Bounds Check Bypass Attack.

```
// i: attacker-controlled index
// buffer_size: valid index bound
// buffer: sensitive data array
// array: attacker-controlled mapping
if (i < buffer_size)
    result = buffer[array[i] * L1];
```

In this example, although the speculative memory access is squashed when the bounds check resolves negatively, the accessed sensitive data is transiently loaded into the cache. Subsequently, the attacker can perform side-channel analysis, such as *Flush+Reload* [52] or *Prime+Probe* [51], to infer which cache lines were loaded, thereby recovering the leaked secret data.

2.3 Cache Side-channel Attacks

Cache side-channel timing attacks exploit the execution time differences that arise from a processor’s cache hierarchy to infer sensitive information [29]. Modern processors use caches to bridge the access speed gap between the CPU and

main memory. While caches improve the retrieval speed of data, they introduce potential risks. Cache side-channel attacks exploit the access time difference between cached and non-cached data. An attacker measures the access time of a specific memory location. Faster access indicates that the data is already in the cache (a “cache hit”). If the data is not in the cache (a “cache miss”), the attacker encounters slower access time since the data is fetched from higher-level caches or main memory. The adversary observes these timing variations to infer whether specific cache lines were accessed, revealing sensitive information.

Typical cache side-channel attacks include *Flush+Reload* [52], *Evict+Time* [17], and *Prime+Probe* [51]. *Flush+Reload* leverages cache maintenance instructions to explicitly flush specific memory locations. By precisely timing subsequent reloads, the attacker can determine whether the victim has accessed the targeted memory based on the measured latency. In contrast, *Evict+Reload* achieves the same objective without relying on flush instructions, instead systematically evicting cache lines through repeated access to congruent addresses within the same cache set, thereby enabling the attacker to monitor the victim’s memory access behavior. *Prime+Probe* differs fundamentally in that it does not require shared memory between the attacker and the victim; rather, the attacker primes the cache by filling it with their own data and then probes it to detect evictions caused by the victim’s accesses, which manifest as measurable variations in the timing of accesses to the probed cache lines.

While cache side-channel attacks are well-studied, speculative execution can also leak information through other microarchitectural structures, including TLBs, store buffers, and load queues, via observable transient states. This broader attack surface motivates cross-component defenses, as pursued by SPLASH.

3 SPLASH Design

3.1 Threat Model and Design Scope

We assume that all speculative instructions are potentially unsafe until they are retired, thereby encompassing both known and emerging speculative execution attack vectors. Our threat model assumes a strong adversary capable of exploiting microarchitectural covert channels, including, but not limited to, caches, memory hierarchies, register files, and branch predictors. The attacker can operate within the same thread (intra-thread) or from a concurrent execution context, such as a sibling thread on the same physical core via Simultaneous Multithreading (SMT). Importantly, we assume the presence of at least one trusted privileged system component, such as machine-mode firmware, a hypervisor, or applications running inside a Trusted Execution Environment (TEE), which forms the Trusted Computing Base (TCB) responsible for enforcing runtime reconfiguration policies of SPLASH. This assumption aligns with the existing literature [6, 14].

SPLASH is designed to defend against speculative execution attacks that produce observable architectural or microarchitectural state changes, such as

modifications to register files [31], cache lines [22], and internal buffers including store buffers and load queues [30]. We explicitly exclude attacks that rely on indirect or non-state-based side channels, such as execution port contention [8] or power-based leakage induced by speculation [13], as mitigating these threats typically requires orthogonal techniques beyond the scope of this work.

3.2 High-level Overview

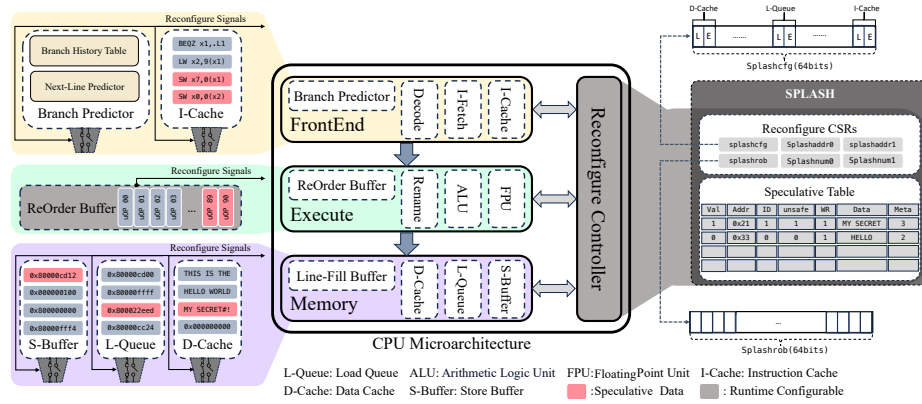


Fig. 1: Overview of the SPLASH Framework

Prior work has explored a range of mitigation strategies to address speculative execution vulnerabilities in modern processors. Software-based defenses, such as the insertion of speculative execution barriers and **FENCE** instructions, can be applied to mitigate these attacks but typically incur substantial performance penalties [49, 50]. Existing hardware-based countermeasures are often rigid and narrowly scoped, targeting only specific classes of known speculative execution attacks and providing protection for a limited subset of CPU microarchitectural components [12, 26]. Consequently, they may not be generalizable to emerging attack vectors. We provide a detailed discussion and comparison of these prior defenses in Section 9.

Unlike prior approaches that target individual speculative execution attacks in isolation, SPLASH addresses speculative leakages at their root by tracking and controlling speculative state at runtime. Figure 1 presents an overview of the SPLASH framework and its integration into a modern out-of-order processor. As illustrated, SPLASH enables runtime-reconfigurable speculative state tracking across the frontend, execution core, and memory subsystem. The depicted pipeline follows a conventional organization, including the branch predictor and instruction cache in the frontend, the ROB and execution units in the execution core, and the load queue, store buffer, and data cache in the memory system.

These structures are common targets of speculative execution attacks, as mis-speculative instructions may transiently update their internal microarchitectural state.

To achieve this, SPLASH introduces Speculative Information Flow Tracking (SIFT), a novel, runtime mechanism that continuously monitors and controls speculative information flows. Specifically, SIFT tracks information associated with speculative instructions, including unique identifiers and updates to speculative state. The tracking is performed by propagating this information across pipeline stages and microarchitectural buffers (detailed in Section 3.3). SIFT, realized by the Speculative Table (Spec-Table), is fully reconfigurable, allowing the system to adapt its protection strategy based on current security requirements dynamically. Specifically, the Spec-Table offers: (i) fine-grained visibility into which microarchitectural elements have been modified speculatively; (ii) the ability to selectively restore original data on misprediction, rather than flushing entire structures; and (iii) efficient hardware integration with minimal impact on pipeline critical paths. Spec-Table is programmed via reconfiguration CSRs to specify which hardware components and signal paths should be protected. This includes security-critical elements of the CPU microarchitecture, such as the branch predictor in the frontend, ROB in the execution unit, and the line-fill buffer in the memory subsystem. Each of these elements has been selected for protection because it represents a critical microarchitectural location where speculative instructions can leave observable traces that attackers exploit. Additionally, the speculative window length, bounded by the ROB head-tail distance, can be tuned at runtime via CSR-based configuration to balance performance and security: shorter speculative windows reduce the exposure window but may increase pipeline stalls, while longer windows improve throughput at potentially higher leakage risk.

Upon the start of a speculative window, SIFT assigns a unique identifier to the speculative window, referred to as *Spec-ID*. Then SPLASH leverages SIFT to track speculative data movements, such as memory loads or any microarchitectural changes made during the speculative window (highlighted with  in Figure 1). SPLASH manages this tracking with the Spec-Table, detailed in Section 3.5, which records the flow of speculative data. During speculative execution, modifications to protected microarchitectural locations are permitted to proceed normally, but the original data is saved in the Spec-Table. If the speculation resolves correctly, the corresponding entries in the Spec-Table are invalidated. In the case of a misprediction, SPLASH restores the prior data, rolling back any speculative modifications for the specific location. Importantly, this recovery process occurs without introducing stalls into the processor pipeline, thereby preserving performance while ensuring security. SPLASH supports runtime reconfiguration through CSRs and a dedicated controller, enabling both reactive and proactive adjustments to protection coverage without disrupting normal operation.

3.3 Speculative Information Flow Tracking

A fundamental challenge in defending against speculative execution attacks is dynamically identifying instructions that execute in the speculative window. Existing approaches often address this issue by tagging instructions [53] or delaying their execution until speculative instructions reach the head of the ROB [37], thereby introducing complex hardware modifications or performance overheads. To address these limitations, SPLASH introduces the SIFT, builds on Spec-ID, a common implementation in out-of-order processors [16, 19], to associate each instruction with its speculative context as it moves through the pipeline. This enables precise tracking of speculative window boundaries on a per-instruction basis. Concretely, for speculative windows initiated by branches or jumps, SPLASH utilizes *branch masks* as Spec-ID to track the speculative state of instructions. When an instruction is fetched, its Spec-ID is updated to reflect the speculative branches (or jumps) it depends upon. For nested branches, multiple bits are set in the Spec-ID. For instance, a branch nested under another will have its bit and its parent’s bit set, representing the nested speculative state. Independent branches have unique bits in the Spec-ID, ensuring that instructions associated with one branch are unaffected by mispredictions of other branches.

As a concrete example, Algorithm 1 contains two branches (**br_1** and **br_2**) and one jump (**j_1**). Assuming the branch predictor always predicts branches and jumps as taken, the instructions following these control flow changes, i.e., lines L8, L12, and L15, are speculatively executed. Their Spec-ID is updated accordingly to reflect their nested relationships. For instance, **br_2** is nested under **br_1**. The instruction at L12 inherits the Spec-ID from **br_1**, updating its Spec-ID from 0000_0001 to 0000_0011 (assuming it is 8-bit). This indicates that it is speculative under both branches.

Algorithm 1 SIFT Demonstration.

```

1: main:
2:   addi x1, x0, 0   # x1 = 0
3:   addi x2, x0, 10  # x2 = 10
4:   beq x1, x1, .L1  # br_1
5:   jal x0, .L3      # j_1
6:
7: .L1:
8:   lw x3, 0(x1)     # Spec-ID = 0000_0001
9:   bne x1, x2, .L2  # br_2
10:
11: .L2
12:   addi x1, x1, 1   # Spec-ID = 0000_0011
13:
14: .L3:
15:   lw x5, 0(x1)     # Spec-ID = 0000_0100

```

When `br_1` is resolved and the prediction is correct, the bit `Spec-ID[0]` is cleared. As a result, L8 is committed, and the `Spec-ID` of L12 becomes `0000_0010`, reflecting that it is still under speculation due to `br_2`. If `br_1` is mispredicted, the speculative instructions dependent on it, such as those at L8 and L12, are flushed from the pipeline.

Note that SIFT tracks speculative *execution context*, rather than multiple secrets or value-level information flow. `Spec-ID` is not a taint label for secret data; instead, it identifies the speculative window to which an instruction belongs. In our implementation, `Spec-ID`s reuse the processor’s existing branch-mask mechanism, so their width is naturally bounded by the maximum number of unresolved control-flow instructions and therefore by the effective ROB capacity. Once a branch is resolved, the corresponding bit is cleared and can be reused, preventing tag exhaustion.

3.4 Eliminate Residual State

As discussed in Section 2, the root cause of speculative execution attacks lies in information leakages that occur during the speculative window. These leakages persist because the microarchitecture state does not fully roll back or flush speculative state when it detects a mis-speculation. Eliminating these residual states is challenging for two main reasons. First, the number of speculative updates can be as large as the length of the speculative window, requiring any protection mechanism to track and store a significant amount of state across multiple components. Second, these updates occur in different microarchitectural components. As a result, a static protection scheme would either have to focus on specific, known attacks, limiting its coverage, or attempt to protect all stateful components, increasing complexity and performance overhead.

SPLASH introduces a novel runtime-reconfigurable protection mechanism to address these challenges. It tracks information flow across the full length of a speculative window, which can be applied to any component in the design, thereby offering comprehensive protection. At the same time, it remains flexible: the speculative window length can be dynamically adjusted based on the trust level of applications and users. Additionally, the mechanism can be configured at runtime to protect only those security-critical components for a given workload selectively. Specifically, we introduce four types of CSRs:

- *splashrob* configures the speculative window length by specifying ROB head-tail distance. This value determines the number of tracked in-flight instructions for potential rollback.
- *splashcfg*, whose each pair of bits corresponds to a specific microarchitectural component, provides fine-grained control over SIFT tracking. Each component can be independently enabled, disabled, or locked, allowing flexible and secure protection configuration at runtime.
- *splashaddr_i* defines instruction memory regions to which SPLASH protection applies. Each entry is encoded using the *Naturally Aligned Power-of-Two* (NAPOT) format: the size of the protected region is inferred from the

number of trailing one bits, while the remaining upper bits define the aligned base address.

- $splashnum_i$ defines the number of SPLASH tracking entries allocated to specific components (e.g., D-Cache and TLB), enabling a tunable trade-off between protection coverage and performance overhead. The configured number of tracking entries should be aligned with the speculative window defined by `splashrob` to ensure that all speculative updates can be safely tracked and restored.

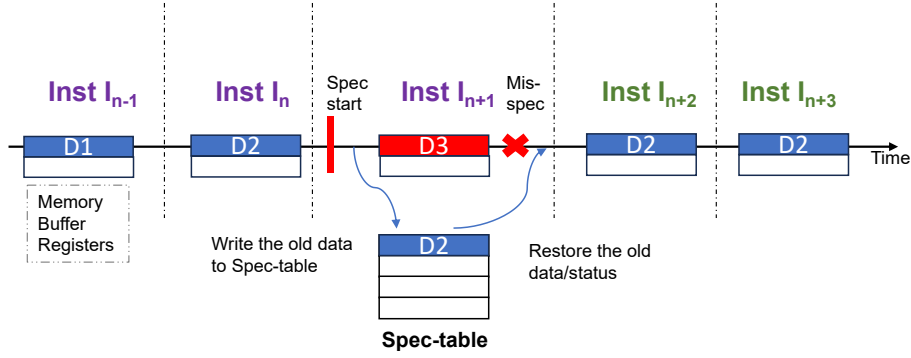


Fig. 2: Rollback of Speculative State Using the Speculative Table.

Furthermore, SPLASH introduces a restore strategy that reverts affected hardware components to their secure pre-speculation state. This ensures that no residual data remains accessible post-misspeculation. For example, SPLASH restores the L1 D-Cache lines affected by the mis-speculated instruction sequence to mitigate Spectre-style cache attacks [2, 22, 23]. This restore mechanism addresses cache-based attacks, including *Prime+Probe* [51] and *Flush+Reload* [52] variants, by eliminating observable differences. As illustrated in Figure 2, instructions I_{n-1} through I_{n+1} modify the microarchitectural state (e.g., by loading data from a lower-level cache or updating buffers or registers). When speculative execution begins at I_{n+1} , any prior state that may be overwritten is captured and saved in the Spec-Table (e.g., data D2). If a misprediction is later detected, SPLASH uses the recorded entry in the Spec-Table to restore the affected hardware component to its pre-speculation state, eliminating all modified traces. Subsequent instructions, such as I_{n+2} and I_{n+3} that merely access the same location without altering its state, continue execution with the correct data after the restore.

3.5 Speculative Table

SPLASH introduces a dedicated hardware mechanism called the Speculative Table (Spec-Table), which enables fine-grained monitoring and control of spec-

ulative execution at the data level. The Spec-Table provides a logically unified abstraction for speculation tracking, while its physical implementation can be distributed across major microarchitectural components, such as the frontend, execution core, and memory hierarchy. Within each of these components, a Spec-Table instance can be shared among the associated subcomponents, providing localized and component-aware protection against speculative execution attacks.

The design of the Spec-Table addresses several core challenges. First, speculative instructions often span multiple pipeline stages and microarchitectural buffers simultaneously, requiring state tracking that is both granular and coherent. Second, reverting speculative effects without halting the pipeline demands precise bookkeeping of data and instructions. Finally, supporting dynamic reconfiguration and selective protection requires a flexible entry format that can represent heterogeneous components and operations. To meet these requirements, each Spec-Table entry incorporates a set of fields that collectively enable accurate monitoring, fast detection of unsafe speculation, and efficient recovery. We describe each field below, along with its motivation and role:

- *Val (Valid)*: A one-bit flag indicating validity of respective entry.
- *Addr (Address)*: Records the address or unique identifier of the in-flight instruction, allowing precise location of the entry in corresponding design components.
- *Spec-ID*: Tracks the Spec-ID associated with the speculative instruction, enabling SIFT precisely.
- *Unsafe*: A one-bit flag that signals the occurrence of mis-speculation, marking potential vulnerabilities that could lead to side-channel attacks if left unaddressed. When mis-speculation occurs, this flag is set, preventing further writes to the associated location.
- *WR (Write)*: A one-bit flag denoting whether a speculative write operation has occurred to a specific location. If a speculative write occurs before mis-speculation is detected ($Unsafe == 1$), this flag is set; otherwise, it remains unset.
- *Data*: Captures the original data value in a buffer, register, or cache line before a speculative write. This enables SPLASH to revert the component to its pre-speculation state if mis-speculation occurs.
- *Meta*: This field logs the metadata prior to speculative changes for components like D-Cache, which rely on metadata such as coherence states.

Together, these fields form a compact but expressive representation of the speculative state. They enable SPLASH to achieve three objectives: (i) early detection and containment of unsafe speculation, (ii) accurate attribution and rollback of speculative effects without pipeline stalls, and (iii) flexible protection policies that adapt to evolving threat models and performance requirements. From a scalability perspective, the Spec-Table grows with the configured speculative window rather than with the semantic complexity of tracked data. Because SPLASH leverages existing speculation metadata (i.e., Spec-IDs derived from branch masks), it does not require separate tags for multiple secrets or

value-level taint domains. As a result, the tracking complexity remains aligned with the number of in-flight speculative updates that must be recoverable, which is bounded by the configured speculative window.

3.6 Flexible Runtime Reconfiguration of SPLASH

The decision to configure SPLASH is delegated to a trusted privileged software context, consistent with the threat model described in Section 3.1. Depending on the platform privilege model, this trusted context may correspond to machine-mode firmware, a hypervisor, or another secure runtime component. SPLASH supports both reactive and proactive reconfiguration. In reactive reconfiguration, if a newly disclosed attack targets a microarchitectural component not previously protected, the configuration can be updated to extend coverage to that component. Alternatively, proactive reconfiguration enables SPLASH to provide broader protection during the execution of sensitive workloads or in higher-assurance modes, regardless of whether a specific exploit has been identified by activating the protection on all components at the cost of performance.

The Reconfigure Controller is exposed through multiple privileged CSRs. These CSRs follow the standard RISC-V privilege model and are accessible only to privileged software. They are not process-specific architectural state and therefore do not require save/restore during ordinary context switching. To clarify the operation of the SPLASH protection mechanism, consider the Spectre attack scenario [22], in which a trusted application exposes an interface to untrusted users (e.g., the Linux kernel).

To mitigate such attacks, the system administrator reconfigures the protection mechanism as follows. First, the administrator identifies the address range of critical code that must be protected against speculative execution. A typical example is an `if-else` sequence [2, 22] that validates privilege before accessing sensitive memory. The identified address range is then programmed into the *splashaddr_X* registers. For instance, if the protection range spans from `0x0000000080000000` to `0x0000000080000100` and the administrator chooses to use *splashaddr₀*, the range is first encoded in NAPOT format (see Section 3.4) and then written as `0x000000002000003F` into *splashaddr₀*. Next, the administrator constrains the size of the ROB, thus limiting the length of the speculative window, by configuring the relevant bits in *splashrob*. In parallel, the administrator programs *splashnum₀* to specify the number of Spec-Table entries allocated to the protected microarchitectural component, which should be consistent with the length of the speculative window. In addition, the administrator specifies which components are protected by setting the appropriate bits in *splashcfg* (see Figure 1). After these configurations are applied, whenever execution in the Linux kernel reaches the protected address range, the SPLASH mechanism automatically restricts the ROB length so that all speculative updates fit within the Spec-Table. In the event of a mis-speculation, all modifications within the protected components are rolled back, ensuring that no microarchitectural information is leaked.

Importantly, the Spec-Table itself is a hardware-internal structure: it is not memory-mapped and is not directly writable by software. Software can only configure protection policy indirectly through the privileged CSRs; the tracking, recording, invalidation, and rollback of speculative state are performed entirely by hardware. Thus, SPLASH provides hardware-enforced speculative protection, while policy decisions, such as when and where protection should be enabled, are handled by trusted system software.

This process is simple and fast, as reconfiguration involves updating only a small set of configuration registers, or if necessary, flushing the state. The *splashcfg* CSR is structured so that each field corresponds to a specific microarchitectural component. Multiple components can be tracked simultaneously by enabling the relevant bits, and configurations targeting different resources can be combined without conflict, since each component’s speculative state is managed independently within the same Spec-Table. Therefore, SPLASH provides flexible configuration capabilities that can be adjusted both reactively and proactively.

4 SPLASH Implementation

SPLASH is a modular and extensible framework designed to protect any microarchitectural component susceptible to speculative execution attacks, including store buffers, fill buffers, branch predictors, and beyond, as discussed in Section 7. Its protection mechanisms are universally applicable and can be re-configured at runtime via CSRs, enabling flexible deployment across different CPU modules without architectural changes.

4.1 SPLASH for D-Cache

We first implement SPLASH for the L1 data cache (D-Cache) as it is a primary target in many real-world speculative execution attacks. In particular, cache-based Spectre variants [22, 30] exploit the fact that transiently executed instructions can leave observable microarchitectural traces in the data cache. Protecting the D-Cache, therefore, mitigates one of the most heavily exploited leakage channels.

In the D-Cache, speculative load instructions that incur cache misses may leave residual microarchitectural traces in the cache hierarchy, which can be exploited by cache-based side-channel attacks. To mitigate such leakage, SPLASH introduces a configurable D-Cache protection mechanism. Specifically, a trusted privileged software component enables D-Cache protection via *splashcfg* register, configures the speculative window through *splashrob*, and, if necessary, defines protected memory regions using *splashaddr_i* along with the corresponding number of SPLASH tracking entries *splashnum_i*.

During execution, consider a speculative load instruction that triggers a cache miss, and its associated Spec-Table is non-zero, indicating it is nested within unresolved control-flow speculation. SPLASH allocates an available entry in the Spec-Table to track this access. For this purpose, it selects an entry with the

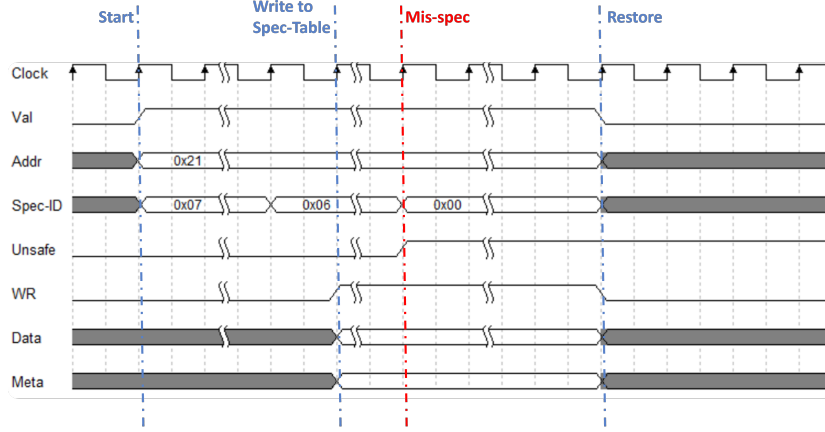


Fig. 3: Waveform of Mis-speculation Handling with $Unsafe == 1$ and $WR == 1$, generated by TimeGen [48].

Val flag set to 0 (i.e., currently unused). If the requested speculative load address matches a valid Spec-Table entry, SPLASH merges the requests to avoid duplication and optimize resource utilization. For the allocated entry, the Val field is set to 1, indicating that the entry is valid. The $Addr$ field records the target address, and the $Spec-ID$ field stores the current speculative context. On resolution of control-flow instructions (e.g., branches or jumps), SPLASH dynamically updates the $Spec-ID$ using branch resolution signals (e.g., **brupdate** in the BOOM processor [54]), ensuring accurate tracking of the speculative window. If the instruction is determined to be mis-speculated, SPLASH sets the $Unsafe$ flag to 1, marking the entry as potentially security-sensitive. Meanwhile, suppose data from a lower-level cache or main memory arrives and is written into the cache line before the instruction is flagged as unsafe. In that case, the WR flag is set to 1, indicating that the cache line has been modified. In this case, SPLASH stores the previous data ($Data$) and metadata ($Meta$) into the Spec-Table entry, enabling future rollback. Conversely, if the $Unsafe$ flag is set before the data is returned, the WR flag remains 0, preventing the data from being written into the cache line. This conservatively blocks speculative data and previous data propagation, reducing unnecessary data movement and improving energy efficiency.

Next, SPLASH evaluates the $Unsafe$ and WR flags to determine the appropriate action:

- $Unsafe == 0$ and $WR == 1$: The instruction was correctly speculated and modified the cache line. SPLASH invalidates the corresponding Spec-Table entry.
- $Unsafe == 1$ and $WR == 0$: The instruction was mis-speculated, but the cache line remains unmodified. In this case, SPLASH bypasses the restore process and invalidates the Spec-Table entry.

- *Unsafe* == 1 and *WR* == 1: The instruction was mis-speculated, and the cache line contains speculative data. SPLASH restores the original data (*Data*) and metadata (*Meta*) to the cache line to prevent any speculative state from being exposed. In the meantime, it invalidates the Spec-Table entry.

Note that an evicted cache line from the L1 D-Cache is forwarded to the write-back buffer only in the case of correct speculation where *Unsafe* == 0 and *WR* == 1. In this scenario, the data and metadata written back are taken from the preserved *Addr*, *Data* and *Meta* fields in the Spec-table. This ensures that no speculative or potentially incorrect state is propagated to the cache hierarchy, thereby preserving architectural correctness and preventing speculative data from contaminating lower levels of the memory system. The detailed waveform in Figure 3 illustrates this process step by step, showing how the Spec-Table fields transition across allocation, speculative updates, mis-speculation detection with *Unsafe* == 1 and *WR* == 1, and eventual restore operations.

4.2 SPLASH for More Microarchitectural Components

SPLASH is designed as a modular framework and can be applied to a wide range of microarchitectural structures that are susceptible to speculative-state pollution. To demonstrate this extensibility, we implement SPLASH support for two representative components beyond the D-Cache: the TLB and the branch predictor. These components are chosen because they are both common targets of speculative execution attacks and differ substantially in their internal state and update semantics, thereby illustrating the generality of the SPLASH design. Both extensions are controlled via CSRs and can be enabled or disabled independently at runtime.

TLB. Wrong-path execution can pollute the TLB by inserting new translations or evicting useful ones. SPLASH leverages the Spec-Table to track speculative TLB refills that overwrite existing entries in a sectored TLB. Upon a speculative load miss, the Spec-Table records the associated Spec-ID along with the original state of the victim entry, including the refill virtual page number (VPN), the page-table level, the victim entry index, and a backup of the evicted translation (tag and level information, as well as per-sector metadata such as the physical page number (PPN) and permission/attribute bits). When a branch misprediction or speculative failure is detected, the backed-up victim entry is restored into the TLB, effectively rolling back the speculative refill and preserving the correct architectural state.

Branch predictor. For the branch predictors, we wrap the predictor with the Spec-Table to snapshot the original state, including the fetch PC, global history, and local history. Upon detecting a branch misprediction, the Spec-Table provides the corresponding snapshot, allowing the front end to temporarily override the history registers and restore the correct predictor state. This mechanism prevents wrong-path execution from permanently perturbing the predictor state while preserving the original commit-based update policy of the BOOM predictor.

5 Evaluation and Benchmark

In this section, we conduct empirical studies to evaluate SPLASH’s effectiveness in mitigating speculative execution attacks, then assess its computational and resource efficiency with different core configurations. We adopted Chipyard [5] v1.10.0 to generate the RTL for BOOM [54] processors, which served as the baseline for comparison. For behavioral simulation, we utilized Synopsys VCS [41] v2023.12 to simulate the modified design. We employed Verdi [42] v2023.12 for waveform analysis to ensure the correctness of the SPLASH implementation.

We integrated SPLASH into the BOOM RTL and performed cycle-accurate simulations using a RISC-V benchmark suite and representative speculative execution attacks. In all evaluated cases, SPLASH successfully prevented speculative leakage through protected components while preserving correct architectural execution. For example, mis-speculative loads left no residual cache state when SPLASH was enabled, whereas the unprotected baseline exhibited clear leakage, confirming that speculative updates are precisely rolled back on misprediction.

For timing and performance comparison, all evaluated designs were synthesized using the same toolchain, identical clock constraints, and the same target platform. Under these conditions, no design required frequency derating, and total cycle count therefore directly reflects normalized execution time, which is standard practice in architecture-level evaluation. We also performed post-synthesis timing analysis under ASIC-style timing constraints through the Chipyard-based synthesis flow to assess the timing characteristics of the SPLASH-enhanced BOOM design. The analysis indicated successful timing closure across target operating points ranging from 500 MHz to 1 GHz. These figures do not correspond to measured operating frequencies on the VC707 FPGA board, which was used only for functional validation and FPGA resource utilization measurement. In the synthesis timing reports, SPLASH-related logic did not appear on the worst timing paths.

Overall, SPLASH provides a runtime-reconfigurable protection surface that maintains near-baseline performance at practical speculative windows ($\text{ROB} \geq 16$), while enabling stronger protection for high-risk code regions through dynamic window reduction without requiring core redesign.

5.1 Vulnerability Mitigation

By design, SPLASH mitigates a broad class of speculative execution attacks by precisely tracking speculative behavior at the data level and eliminating residual microarchitectural state introduced by mis-speculated instructions. To validate this capability, we evaluate SPLASH against three representative and widely studied attacks, Bounds Check Bypass (BCB), Branch Target Injection (BTI), and Speculative Store Bypass (SSB), on a SPLASH-protected BOOM processor. To our knowledge, they are the only speculative execution vulnerabilities that concretely impact the BOOM processor in practice [9, 12, 19]⁵. As expected,

⁵ We exclude Return Stack Buffer (RSB) attack in our evaluation, as it is disconnected from the Branch Prediction Unit (BPU) in the current BOOM implementation [9].

experimental results confirm that SPLASH successfully mitigates all attacks, ensuring that sensitive information remains secure and inaccessible through speculative side channels. Although this evaluation focuses on the L1 D-Cache, the SPLASH architecture is designed to generalize across other vulnerable microarchitectural components, such as store buffers and branch predictors. Indeed, by leveraging a unified SIFT mechanism and a configurable Speculative Table, SPLASH can similarly monitor and mitigate speculation-induced vulnerabilities in these subsystems, offering comprehensive, pipeline-wide protection against speculative execution attacks.

5.2 Performance Benchmark

We evaluate the performance impact of SPLASH on two BOOM core configurations using a RISC-V benchmark suite. To ensure a fair and consistent comparison with state-of-the-art speculation mitigation schemes, we selected and adapted SpecLFB [12], SpecTerminator-v1 [19], SpecTerminator-v2 [19], NDA [46], and SSE-RV [33] to a common RTL baseline compatible with Chipyard v1.10.0⁶. Unless explicitly stated otherwise, SPLASH is configured with only D-cache protection enabled in the performance evaluation, while protections for other components such as the TLB and branch predictor are disabled. This matches the protection scope of the baseline mitigation techniques considered here, which target cache-based speculative leakage channels, and therefore ensures a fair comparison. Execution time is measured as the total number of cycles required to complete each benchmark and is used as the primary metric for performance evaluation. In addition to the quantitative performance comparison reported below, it is important to note that the baseline mitigations considered here are fixed-function designs and do not provide runtime reconfigurability comparable to SPLASH.

5.2.1 Small BOOM

We first conduct a performance benchmark with a small BOOM core; the execution time is normalized to the unprotected baseline BOOM core with a ROB size of 32 entries. For SPLASH, we evaluate two configurations by varying the ROB head-tail distance to 16 and 32, denoted as SPLASH-ROB16 and SPLASH-ROB32, respectively. Following the original designs of the baseline schemes, SpecLFB, SpecTerminator-v1/-v2, NDA, and SSE-RV are evaluated with a fixed ROB size of 32. Table 1 summarizes the normalized execution time across six representative benchmarks, spanning vector operations (*vvadd*, *mt-vvadd*), sorting (*rsort*), compute-intensive workloads (*multiply*, *dhrystone*), and a memory-intensive matrix multiplication (*mt-matmul*).

⁶ The RTL implementation of SpecLFB was adapted from the code released by the authors of [12]. The implementations of SpecTerminator-v1/v2, NDA, and SSE-RV were adapted from code previously released online by the authors of SpecTerminator [19].

Table 1: Normalized execution time across different benchmarks for various speculation mitigation schemes on small BOOM [54] core. For SPLASH, the reported results use D-cache protection only. **SpecT.** stands for **SpecTerminator**.

	vvadd	rsort	mt-vvadd	multiply	mt-matmul	dhystone	Avg
SPLASH-ROB16	0.07%	0.47%	0.00%	0.07%	0.02%	0.00%	0.11%
SPLASH-ROB32	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
SpecT-v1	0.00%	0.00%	0.00%	0.07%	0.00%	0.00%	0.01%
SpecT-v2	0.00%	0.00%	0.00%	0.07%	0.00%	0.00%	0.01%
SpecLFB	1.71%	4.41%	0.44%	2.26%	27.75%	2.22%	6.46%
NDA	23.00%	73.00%	52.00%	59.00%	67.00%	42.00%	52.67%
SSE-RV	15.00%	7.00%	8.00%	17.08%	23.00%	19.00%	14.85%

Table 2: Normalized execution time across different benchmarks for various speculation mitigation schemes on Medium BOOM [54] core. For SPLASH, the reported results use D-cache protection only. **SpecT.** stands for **SpecTerminator**.

	vvadd	rsort	mt-vvadd	multiply	mt-matmul	dhystone	Avg
SPLASH-ROB16	0.85%	8.49%	2.13%	0.18%	0.42%	6.64%	3.12%
SPLASH-ROB32	0.31%	0.79%	0.00%	0.09%	0.02%	0.15%	0.23%
SPLASH-ROB64	0.17%	1.37%	0.28%	0.01%	0.00%	0.13%	0.33%
SpecT-v1	0.68%	6.92%	2.01%	0.18%	0.00%	4.92%	2.45%
SpecT-v2	0.68%	6.92%	1.92%	0.18%	0.00%	4.92%	2.44%
SpecLFB	0.29%	0.66%	0.03%	2.44%	5.26%	0.25%	1.49%
NDA	30.89%	92.46%	63.22%	58.29%	66.00%	48.99%	59.98%
SSE-RV	15.78%	14.41%	12.11%	18.22%	23.00%	24.86%	18.06%

The results demonstrate that SPLASH achieves near performance-neutral execution when sufficient speculative tracking capacity is available. In particular, SPLASH-ROB32 introduces zero overhead across all benchmarks, matching the baseline performance. Even when the speculative window is reduced to 16 entries, SPLASH-ROB16 maintains excellent performance, incurring an average slowdown of only 0.11%.

Notably, SPLASH-ROB16 consistently outperforms SpecLFB across all workloads, and avoids the severe slowdown observed for SpecLFB on the memory-intensive *mt-matmul* benchmark (27.75% for SpecLFB versus 0.02% for SPLASH-ROB16). On average, SPLASH achieves 129.2 $\times$ better timing performance compared to SpecLFB, the best-performing design in the literature. The key reason is that SPLASH preserves the processor’s steady-state speculation behavior: it does not delay speculative loads, insert speculation barriers, or otherwise serialize the normal speculative execution path. Instead, SPLASH augments the processor with lightweight metadata tracking and selective rollback mechanisms.

SpecTerminator-v1 and SpecTerminator-v2 exhibit minimal execution-time overhead on most benchmarks, consistent with their goal of aggressively terminating misspeculation with limited steady-state impact. However, this comes at the cost of hardware overhead (discussed in Section 5.3). In contrast, NDA and SSE-RV impose substantial performance penalties, with NDA exhibiting slowdowns exceeding 50% on average and SSE-RV incurring notable overhead, particularly on memory-intensive workloads.

5.2.2 Medium BOOM

Next, we benchmark the performance on the Medium BOOM core, with all results normalized to the unprotected baseline configuration using ROB=64. As shown in Table 2, the results align with those reported for the small core, while also highlighting the scalability of SPLASH to deeper pipelines and larger speculation windows.

Concretely, SPLASH continues to exhibit well-controlled performance overhead when configured with practical ROB head-tail distances, for the same reason observed on the small core: when the configured speculative window remains close to the baseline ROB capacity, SPLASH preserves the processor’s normal speculative behavior and introduces only lightweight tracking overhead. In particular, SPLASH-ROB32 and SPLASH-ROB64 remain very close to the baseline across all benchmarks, incurring average slowdowns of only 0.23% and 0.33%, respectively. These configurations consistently outperform SpecTerminator-v1 and SpecTerminator-v2 (2.45% and 2.44% average overhead) as well as SpecLFB (1.49% average overhead), demonstrating that SPLASH scales more favorably as core complexity increases. The advantage of SPLASH is especially evident on memory- and speculation-intensive workloads, where its performance remains near baseline while other schemes incur noticeable slowdowns. When the speculative window is more aggressively constrained (SPLASH-ROB16), SPLASH incurs a higher but still negligible average overhead of 3.12%, with slowdowns primarily observed on *rsort* and *dhrystone*. This behavior reflects an expected and controllable trade-off between limiting speculative execution depth and maintaining throughput, rather than inefficiencies inherent in the SPLASH design. Consistent with prior observations, NDA and SSE-RV impose substantial execution time overhead across all benchmarks, with average slowdowns of 59.98% and 18.06%, respectively, indicating significantly higher performance costs in exchange for their protection mechanisms. However, on average across all configurations on medium BOOM, SPLASH incurs a 1.23% slowdown, which is $1.21\times$ better than the best-performing related work, SpecLFB, which introduces a 1.49% slowdown. Unlike these prior schemes, which are statically bound to fixed mitigation logic once implemented, SPLASH additionally provides runtime reconfigurability, allowing the protection scope and speculative window to be adapted to workload requirements without hardware redesign.

5.3 Hardware Overhead

To evaluate the hardware resource overhead introduced by SPLASH, we synthesized FPGA implementations of the baseline BOOM processor and its SPLASH-protected variants for the small-core configuration. For a fair comparison, we also selected and adapted SpecLFB [12], SpecTerminator-v1 [19], SpecTerminator-v2 [19], NDA [46], and SSE-RV [33] to the same Chipyard v1.10.0 framework. Unless explicitly stated otherwise, the reported SPLASH resource overhead corresponds to the same D-cache protection configuration used in the performance evaluation, with protections for other components disabled. This matches the protection scope of the baseline cache-oriented mitigations and avoids biasing

Table 3: Normalized LUT and FF utilization across speculation mitigation schemes. SPLASH subcolumns denote Spec-Table entries and SpecT. stands for SpecTerminator.

Resource	Baseline	SPLASH (our work)				SpecT. (v1)	SpecT. (v2)	SpecLFB	NDA	SSE-RV
		4 Entry	8 Entry	16 Entry	32 Entry					
# LUTs (Overhead %)	91310 (0.00%)	92982 (1.83%)	94626 (3.63%)	97905 (7.22%)	104037 (13.94%)	120607 (32.08%)	123466 (35.21%)	93409 (2.30%)	120788 (32.28%)	110923 (21.47%)
# Flip-Flops (Overhead %)	64257 (0.00%)	64436 (0.28%)	64530 (0.42%)	64741 (0.75%)	65134 (1.36%)	67415 (4.91%)	67699 (5.35%)	64684 (0.66%)	65969 (2.66%)	69788 (8.61%)

the area comparison in favor of SPLASH. All designs were deployed on the Xilinx VC707 FPGA board, which is equipped with a Virtex-7 XC7VX485T FPGA. We used Vitis [4] v2023.2 as the synthesis and implementation toolchain. All designs were implemented under an identical toolchain and evaluation flow. This ensures that the reported resource numbers reflect architectural differences rather than tooling or configuration artifacts.

A key design goal of SPLASH is to expose protection strength as a tunable hardware parameter. Accordingly, we implement Spec-Table with sizes ranging from 4 to 32 entries, where 32 corresponds to the maximum ROB length of the small core (i.e., the widest reasonable speculative window on this design). This sweep allows us to evaluate how SPLASH scales in hardware cost as the speculative window and tracking capacity are increased, and to provide practical sizing guidance. We collect detailed FPGA utilization statistics, including LUTs, LUTRAMs, FFs, BRAMs, DSPs, I/O blocks, BUFGs, MMCMs, and PLLs. Across all evaluated designs, only LUT and FF utilization exhibit noticeable variation, while other resources remain nearly unchanged. Therefore, we focus our comparison on LUTs and FFs, which most directly reflect the logic and state overhead of speculation mitigation mechanisms. Note that detailed per-component area/power consumption is not reported here, as the primary contribution of this work is the architectural evaluation and scalability of the reconfigurable mechanism. The reported LUT and FF overhead captures the essential resource impact.

Table 3 summarizes the normalized LUT and FF utilization across all schemes. SPLASH demonstrates a gradual and predictable scaling trend: as the Spec-Table size increases from 4 to 32 entries, LUT utilization increases from 1.83% to at most 13.94% compared to baseline, while FF utilization remains very small (at most 1.36% more). This trend indicates that SPLASH primarily introduces lightweight control logic and metadata tracking. In comparison, SpecLFB (the best performing state-of-the-art) incurs 2.3% more LUT and 0.66% more FF because its protection logic is largely confined to the line-fill buffer and limited to the data cache sub-system. However, this narrow placement limits the scope and flexibility of protection. SpecTerminator-v1/v2 impose significantly higher logic overhead (up to 32.08%–35.21% more LUT and 4.91%–5.35% more FF); NDA and SSE-RV also show increased costs, particularly requiring 32.28% and 21.47% more FF, suggesting register-intensive trade-offs.

Overall, compared to most related work, SPLASH requires significantly less hardware area. The primary exception is SpecLFB: SPLASH uses $2.89\times$ more LUTs and $1.06\times$ more flip-flops than SpecLFB. This overhead stems from two key factors. First, SPLASH provides fully dynamic reconfiguration, enabling flexible security policies and multiple levels of protection—capabilities introduced for the first time and not supported by SpecLFB. Second, unlike SpecLFB, which is limited to the data-cache subsystem, SPLASH is designed to protect a broad range of microarchitectural components, including the TLB and other speculation-relevant structures. Despite this modest increase in area, SPLASH delivers substantially better performance. As discussed in Section 5.2, SPLASH

outperforms prior work by $129.2\times$ on small BOOM and $1.2\times$ on medium BOOM, making the additional hardware cost well justified. Moreover, unlike prior hardware mitigations with fixed protection logic, SPLASH supports runtime-selectable protection policies, providing a flexibility advantage that is not captured by performance and resource metrics alone.

6 Security Analysis

This section discusses SPLASH’s security guarantees under two representative attack scenarios: *shared-thread* attacks, in which attacker-controlled and victim instructions execute in the same thread context, and *cross-thread* attacks, in which speculative side effects propagate through microarchitectural state shared across logical threads. Together, these two scenarios cover the major speculative-execution attack settings considered in prior work [1, 2, 22, 30, 31].

Importantly, the mechanisms described below are general and extend to all transient execution windows, including those triggered by mispredicted branches or exceptions/faults (e.g., Meltdown-style attacks). SIFT assigns unique Spec-IDs to instructions entering a transient window, tracks microarchitectural updates, and ensures rollback upon window resolution, preventing any transient path from leaking information through side channels.

6.1 Shared Thread

A representative shared-thread attack is Bounds Check Bypass (BCB) [22], in which an attacker mistrains a branch predictor so that a bounds check is transiently bypassed and a secret-dependent access updates a microarchitectural structure such as the D-cache. In this setting, the attacker and victim instructions execute within the same speculative window.

SPLASH mitigates this leakage by using SIFT to associate in-flight instructions with their speculative context and by using the Spec-Table to record modifications to protected components during that window. If the branch later resolves as mispredicted, SPLASH invalidates or restores the affected speculative state before it can remain observable. For example, if a wrong-path load transiently fills a cache line that encodes a secret, the previous cache state recorded in the Spec-Table is restored on mis-speculation, preventing the attacker from inferring the secret through a cache side channel. Thus, in the shared-thread setting, SPLASH blocks leakage by ensuring that wrong-path speculative updates do not survive beyond the speculative window.

6.2 Cross Threads

A representative cross-thread setting arises when simultaneous multithreading (SMT) causes logical threads to share microarchitectural resources such as caches, buffers, or predictor state. Attacks such as Zenbleed [31] and Downfall [30] show

that insufficient isolation of transient state can expose information across thread boundaries.

SPLASH addresses this setting by tracking speculative updates to protected shared structures in the Spec-Table, regardless of which logical thread issued the speculative instruction. If a mis-speculation in one thread modifies shared state that could become visible to another thread, SPLASH rolls back the recorded update before it can persist as an observable side effect. This allows SPLASH to prevent cross-thread leakage through protected shared components without requiring thread-private duplication of protection logic.

Overall, SPLASH protects against both same-thread and cross-thread speculative and transient execution leakage by coupling speculative-context tracking with selective rollback of protected microarchitectural state. Because the speculative window is bounded at runtime, the amount of state that must be tracked remains controlled, allowing SPLASH to provide broad protection with low overhead across all transient execution paths.

7 Discussion

7.1 Extension to Transient Execution Attacks

While SPLASH is designed primarily for speculative execution attacks, its architecture generalizes to broader transient execution attacks, including Meltdown [27]. Transient execution refers to instructions that execute temporarily before the architectural state is committed. Speculative execution attacks are a subset, triggered by mispredicted branches or data dependencies, while Meltdown-style attacks exploit faults or exceptions.

SIFT can be extended to handle such transient windows by assigning unique Spec-IDs to instructions entering each transient window. All microarchitectural updates performed during the window are tracked in the Spec-Table. Upon resolution, whether due to a misprediction or an exception, these updates are rolled back, sanitizing the transient state and preventing side-channel leakage. This approach ensures that SPLASH enforces correct speculative/transient tracking for all in-flight instructions, providing a rigorous mechanism that covers all transient execution paths that could lead to observable microarchitectural side effects.

7.2 Resilience against adaptive adversaries

SPLASH is designed to remain effective against adaptive attackers who target different components, modify covert channels, or craft new speculative gadgets to bypass static defenses. Its runtime reconfigurability enables the selective enablement or disablement of protections for specific microarchitectural components through `splashcfg`, with negligible overhead. Moreover, SPLASH can dynamically bound the speculative window using `splashrob`, limiting the amount of speculative state an attacker can influence, which is a vital mitigation capability introduced for the first time. SPLASH also supports selective protection of

security-critical code regions via `splashaddr`. This novel mechanism reduces performance overhead for the rest of the application while making it more difficult for attackers to locate unprotected gadgets. Finally, by jointly tracking speculative state across caches, the TLB, and branch predictors, SPLASH covers the primary covert channels exploited by known classes of speculative-execution attacks. Its unified tracking framework can be extended to incorporate newly discovered microarchitectural attack vectors.

7.3 Applicability to Other Out-of-Order Processors

The core idea behind SPLASH is based on the observation that all out-of-order processors inherently maintain metadata to track whether instructions are under speculation, such as speculation IDs associated with micro-operations. SPLASH leverages these existing signals (e.g., Spec-IDs or their equivalents) to selectively record and later remove speculative state if speculative execution fails [16, 19]. Therefore, the mechanism itself is not tied to any specific pipeline microarchitecture or to the RISC-V BOOM core in particular, and can be adapted to other out-of-order designs, including ARM or x86 processors. As such, we believe SPLASH is broadly applicable to a wide range of architectures, and its integration can be relatively straightforward wherever speculation tracking metadata is already available.

7.4 Performance Overhead During Reconfiguration

SPLASH is reconfigured by updating a small set of privileged CSRs, including `splashcfg`, `splashrob`, `splashaddr`, and `splashnum`. These registers determine which microarchitectural components are protected, the effective speculative-window bound, the protected instruction regions, and the number of Spec-Table entries allocated to each protected component.

To ensure a clean transition between configurations, reconfiguration involves more than a single CSR write. When `splashcfg` or related configuration CSRs are updated, the processor first flushes any in-flight speculative instructions and invalidates all valid entries in the Spec-Table. The new configuration is then applied atomically. As a result, no speculative metadata survives across reconfiguration boundaries, and stale speculative state cannot affect execution under the new protection policy.

Although reconfiguration requires these additional steps, its cost remains low because it is incurred only when the protection policy is explicitly changed, rather than during normal execution. In particular, SPLASH does not introduce any reconfiguration latency on the steady-state execution path. Once configured, SPLASH operates transparently in hardware without further software intervention, and the runtime cost is limited to the lightweight metadata tracking overhead quantified in Section 5.3.

This behavior contrasts with mitigations that rely on repeatedly flushing large microarchitectural structures or introducing continuous execution-time serialization. In SPLASH, reconfiguration is infrequent, bounded, and isolated

from the common-case execution path, preserving the low-overhead nature of the design.

7.5 TEE Independence

SPLASH does not inherently depend on the TEE or the TCB for enforcement. The protection mechanism is implemented entirely in hardware and operates by associating speculative state with speculation epochs tracked in the Spec-Table. These protections can be applied seamlessly in conventional OS environments or hypervisor-managed systems without requiring a TEE. If desired, SPLASH can operate independently of any TEE, and its hardware resource and performance overheads remain unchanged. By contrast, employing a TEE or a minimal TCB can provide additional security guarantees, such as isolation from privileged software. But these considerations are orthogonal to SPLASH contributions.

8 Future Extensions of SPLASH

As discussed in Section 3, the current implementation of SPLASH focuses on mitigating speculative execution attacks and is prototyped on the BOOM processor, which, to the best of our knowledge, is the most sophisticated open-source RISC-V core supporting out-of-order execution and thus a suitable platform for evaluating speculation-based defenses. Although BOOM does not expose the full range of transient execution behaviors present in some commercial processors, the design of SPLASH is not inherently restricted to speculative execution. In particular, the SIFT mechanism can be extended to transient execution by assigning unique Spec-IDs to instructions that initiate transient windows, allowing SPLASH to sanitize transient microarchitectural state using its existing rollback mechanisms upon faults or exceptions. Exploring and validating such extensions on processors with more comprehensive transient execution features remains an important direction for future work.

For similar reasons, the current prototype does not extend SPLASH protection to *all* microarchitectural components. In particular, store buffers are not tracked because, in BOOM, speculative stores are retained in the store queue and only update the D-cache upon instruction commit. As a result, speculative stores are naturally squashed on misprediction and do not introduce a persistent microarchitectural state that could be exploited as a leakage channel. The SPLASH design is not inherently limited to the components evaluated in this study and could be extended to additional structures in other microarchitectural implementations.

Finally, while SPLASH provides adaptable protection against speculative execution attacks, it relies on trusted software to manage its runtime configuration. This assumption is consistent with prior works [6, 14] (detailed in Section 3.1), which similarly depend on trusted components for secure configuration and control. As discussed in Section 7.5, SPLASH is designed to be independent of any specific TEE implementation.

9 Related Work

Existing hardware-based mitigation mechanisms for speculative execution attacks often rely on static strategies tailored to specific, known vulnerabilities targeting certain CPU components. For instance, several works focus exclusively on protecting caches, including Conditional Speculation [26], MuonTrap [3], CleanupSpec [36], and SpecLFB [12]. Slightly broader approaches, such as InvisiSpec [50] and SafeSpec [20], employ shadow buffers to conceal speculative execution results, thereby extending protection to the TLB sub-system and the cache hierarchy.

Another subset of works, such as NDA [46], SpecShield [7], STT [53], and DOLMA [28], delay the propagation of speculative results to block transient side channels. However, while these techniques claim comprehensive protection, they fail to address certain vulnerabilities. For example, STT overlooks TLB side channels [19], and DOLMA does not mitigate load-load reordering side channels [19]. Furthermore, these methods often lack scalability and fail to adapt to emerging speculative attack vectors. SpecTerminator [19], developed for the BOOM processor, introduces two mitigation strategies: SpecTerminator-v1, which targets speculative cache and other memory hierarchy side channels, and SpecTerminator-v2, which expands coverage to all known speculative side channels. However, despite its broader scope, SpecTerminator struggles to address newly discovered speculative execution attacks and remains limited by its static design. SSE-RV [33] employs hardware-level speculative taint tracking to block cache-based covert channels and has been prototyped on real BOOM hardware with protection against multiple Spectre variants. However, its scope is largely limited to cache covert channels, relying on global load fencing and pipeline serialization, which results in limited flexibility and potential performance overhead. Indeed, while these approaches address speculative execution vulnerabilities to varying degrees, they remain narrowly focused and inflexible. Their reliance on static strategies or invasive architectural modifications underscores the pressing need for a more adaptable, scalable, and efficient solution capable of comprehensively addressing speculative execution attacks across diverse processor architectures and emerging threats. Unlike prior efforts, SPLASH introduces a reconfigurable framework that provides fine-grained, system-wide protection against known and future speculative execution attacks by dynamically tracking and sanitizing residual microarchitectural state.

The rollback mechanism in SPLASH is similar to prior work such as CleanupSpec [36], ReversiSpec [47], ReViCe [21], and CacheRewinder [24], all of which aim to revert cache state changes introduced during mis-speculation. However, these mechanisms are generally statically limited to specific microarchitectural components, primarily the data cache, and often require fine-grained tracking across the full CPU pipeline. This results in substantial hardware complexity and runtime overhead. Furthermore, several of these approaches lack generality, offering limited flexibility in defining where and how protections are applied, and may still expose timing-based side channels [32]. In contrast, SPLASH generalizes speculative rollback through SIFT, enabling efficient and precise recovery

across a broader set of microarchitectural components, including caches, buffers, and predictors, while supporting runtime-reconfigurable protection policies. We also note that the term *speculative information flow tracking* has appeared in the patent literature, e.g., US11301591B2 / EP4022484B1 [34, 35]. Compared with that disclosure, which relies on taint propagation and issue/remediation logic to suppress speculative side effects, SPLASH uses SIFT within a runtime-reconfigurable framework that records and restores protected microarchitectural state through the Spec-Table. In addition, SPLASH supports selective region protection and dynamic speculative-window bounding.

10 Conclusion

The SPLASH framework presents a novel and reconfigurable approach to mitigating speculative execution vulnerabilities in modern processors. By combining Speculative Information Flow Tracking with a programmable Speculative Table, SPLASH enables precise tracking and rollback of speculative microarchitectural state, addressing speculative leakage at its root rather than reacting to individual attack variants. Implemented on the BOOM processor, SPLASH demonstrates comprehensive protection across multiple microarchitectural components while incurring only modest hardware overhead and near-zero performance impact, outperforming state-of-the-art defenses such as SpecTerminator in both flexibility and efficiency. Crucially, SPLASH exposes speculative execution control as a runtime-configurable mechanism, allowing security policies to be dynamically adapted to the workload and threat context without requiring hardware redesign. By unifying strong security guarantees with scalability and performance preservation, SPLASH offers a practical and future-proof foundation for defending next-generation processors against both current and emerging speculative execution attacks.

Acknowledgement

Our research work was partially funded by Intel’s Scalable Assurance Program, DFG-SFB 1119-236615297, the European Union under Horizon Europe Programme-Grant Agreement 101070537-CrossCon, NSF-DFG-Grant 538883423, the European Research Council under the ERC Programme-Grant 101055025-HYDRANOS, and Synopsys with EDA tools licenses support. This work does not in any way constitute an Intel endorsement of a product or supplier. Any opinions, findings, conclusions, or recommendations expressed herein are those of the authors and do not necessarily reflect those of Intel, the European Union, or the European Research Council. We also thank the anonymous reviewers for their helpful comments and suggestions.

References

1. Branch target injection. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/branch-target-injection.html>
2. Speculative store bypass. <https://nvd.nist.gov/vuln/detail/CVE-2018-3639>
3. Ainsworth, S., Jones, T.M.: Muontrap: Preventing cross-domain spectre-like attacks by capturing speculative state. In: 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA). pp. 132–144. IEEE (2020)
4. AMD, I.: Amd vitis. <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis.html>
5. Amid, A., Biancolin, D., Gonzalez, A., Grubb, D., Karandikar, S., Liew, H., Magyar, A., Mao, H., Ou, A., Pemberton, N., Rigge, P., Schmidt, C., Wright, J., Zhao, J., Shao, Y.S., Asanović, K., Nikolić, B.: Chipyard: Integrated design, simulation, and implementation framework for custom socs. *IEEE Micro* **40**(4), 10–21 (2020). <https://doi.org/10.1109/MM.2020.2996616>
6. Bahmani, R., Brasser, F., Dessouky, G., Jauernig, P., Klimmek, M., Sadeghi, A.R., Stapf, E.: {CURE}: A security architecture with {CUsomizable} and resilient enclaves. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 1073–1090 (2021)
7. Barber, K., Bacha, A., Zhou, L., Zhang, Y., Teodorescu, R.: Specshield: Shielding speculative data from microarchitectural covert channels. In: 2019 28th International Conference on Parallel Architectures and Compilation Techniques (PACT). pp. 151–164. IEEE (2019)
8. Bhattacharyya, A., Sandulescu, A., Neugschwandtner, M., Sorniotti, A., Falsafi, B., Payer, M., Kurmus, A.: Smotherspectre: exploiting speculative execution through port contention. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 785–800 (2019)
9. BOOM Project Contributors: BOOM Transient Execution Attacks. <https://github.com/riscv-boom/boom-attacks> (2024), accessed: 2024-04-20
10. Canella, C., Genkin, D., Giner, L., Gruss, D., Lipp, M., Minkin, M., Moghimi, D., Piessens, F., Schwarz, M., Sunar, B., et al.: Fallout: Leaking data on meltdown-resistant cpus. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 769–784 (2019)
11. Canella, C., Van Bulck, J., Schwarz, M., Lipp, M., Von Berg, B., Ortner, P., Piessens, F., Evtushkin, D., Gruss, D.: A systematic evaluation of transient execution attacks and defenses. In: 28th USENIX Security Symposium (USENIX Security 19). pp. 249–266 (2019)
12. Cheng, X., Tong, F., Wang, H., Zhou, Z., Jiang, F., Mao, Y.: Specflb: Eliminating cache side channels in speculative executions. Presented at USENIX Security p. 1 (2024)
13. Chowdhury, M.H.I., Zhang, Z., Yao, F.: Powspectre: Powering up speculation attacks with tsx-based replay. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security. pp. 498–511 (2024)
14. Dessouky, G., Gruler, A., Mahmood, P., Sadeghi, A.R., Stapf, E.: Chunked-cache: On-demand and scalable cache isolation for security architectures. *arXiv preprint arXiv:2110.08139* (2021)
15. Faraboschi, P., Fisher, J.A., Young, C.: Instruction scheduling for instruction level parallel processors. *Proceedings of the IEEE* **89**(11), 1638–1659 (2001)

16. Fog, A.: The microarchitecture of intel, amd and via cpus: An optimization guide for assembly programmers and compiler makers. Software optimization resources (2016)
17. Irazoqui, G., Inci, M.S., Eisenbarth, T., Sunar, B.: Wait a minute! a fast, cross-vm attack on aes. In: Research in Attacks, Intrusions and Defenses: 17th International Symposium, RAID 2014, Gothenburg, Sweden, September 17-19, 2014. Proceedings 17. pp. 299–319. Springer (2014)
18. Jang, H., Shin, Y.: Microcfi: Microarchitecture-level control-flow restrictions for spectre mitigation. IEEE Access (2023)
19. Jin, H., He, Z., Qiang, W.: Specterminator: Blocking speculative side channels based on instruction classes on risc-v. ACM Transactions on Architecture and Code Optimization **20**(1), 1–26 (2023)
20. Khasawneh, K.N., Koruyeh, E.M., Song, C., Evtvushkin, D., Ponomarev, D., Abu-Ghazaleh, N.: Safespec: Banishing the spectre of a meltdown with leakage-free speculation. In: 2019 56th ACM/IEEE Design Automation Conference (DAC). pp. 1–6. IEEE (2019)
21. Kim, S., Mahmud, F., Huang, J., Majumder, P., Christou, N., Muzahid, A., Tsai, C.C., Kim, E.J.: Revice: Reusing victim cache to prevent speculative cache leakage. In: 2020 IEEE Secure Development (SecDev). pp. 96–107. IEEE (2020)
22. Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., et al.: Spectre attacks: Exploiting speculative execution. Communications of the ACM **63**(7), 93–101 (2020)
23. Koruyeh, E.M., Khasawneh, K.N., Song, C., Abu-Ghazaleh, N.: Spectre returns! speculation attacks using the return stack buffer. In: 12th USENIX Workshop on Offensive Technologies (WOOT 18) (2018)
24. Lee, J., Lee, J., Suh, T., Koo, G.: Cacherewinder: revoking speculative cache updates exploiting write-back buffer. In: 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE). pp. 514–519. IEEE (2022)
25. Li, L., Yavarzadeh, H., Tullsen, D.: Indirector: {High-Precision} branch target injection attacks exploiting the indirect branch predictor. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 2137–2154 (2024)
26. Li, P., Zhao, L., Hou, R., Zhang, L., Meng, D.: Conditional speculation: An effective approach to safeguard out-of-order execution against spectre attacks. In: 2019 IEEE international symposium on high performance computer architecture (HPCA). pp. 264–276. IEEE (2019)
27. Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Horn, J., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., et al.: Meltdown: Reading kernel memory from user space. Communications of the ACM **63**(6), 46–56 (2020)
28. Loughlin, K., Neal, I., Ma, J., Tsai, E., Weisse, O., Narayanasamy, S., Kasikci, B.: {DOLMA}: Securing speculation with the principle of transient {Non-Observability}. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 1397–1414 (2021)
29. Lyu, Y., Mishra, P.: A survey of side-channel attacks on caches and countermeasures. Journal of Hardware and Systems Security **2**, 33–50 (2018)
30. Moghimi, D.: Downfall: Exploiting speculative data gathering. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 7179–7193 (2023)
31. Ormandy, T.: Zenbleed. <https://bit.ly/zenbleed> (2023)
32. Randal, A.: This is how you lose the transient execution war. arXiv preprint arXiv:2309.03376 (2023)

33. Sabbagh, M., Fei, Y.: Secure speculative execution via risc-v open hardware design. In: Fifth Workshop on Computer Architecture Research with RISC-V (CARRV 2021) (2021)
34. Saileshwar, G., Chowdhury, M.: Speculative information flow tracking. U.S. Patent US11301591B2 (2022), google Patents
35. Saileshwar, G., Chowdhury, M.: Speculative information flow tracking. European Patent EP4022484B1 (2024), google Patents
36. Saileshwar, G., Qureshi, M.K.: Cleanupspec: An” undo” approach to safe speculation. In: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture. pp. 73–86 (2019)
37. Sakalis, C., Kaxiras, S., Ros, A., Jimborean, A., Själander, M.: Efficient invisible speculative execution through selective delay and value prediction. In: Proceedings of the 46th International Symposium on Computer Architecture. pp. 723–735 (2019)
38. Schwarz, M., Lipp, M., Moghimi, D., Van Bulck, J., Smullen, C., Genkin, D., Sunar, B., Gruss, D., Piessens, F., Mangard, S.: Zombieload: Cross-privilege-boundary data sampling. In: Proceedings of the 26th ACM Conference on Computer and Communications Security (CCS). pp. 753–768. ACM (2019)
39. Sharkey, J., Balkan, D., Ponomarev, D.: Adaptive reorder buffers for smt processors. In: Proceedings of the 15th international conference on Parallel architectures and compilation techniques. pp. 244–253 (2006)
40. Silc, J., Robic, B., Ungerer, T.: Asynchrony in parallel computing: From dataflow to multithreading. *Parallel and Distributed Computing Practices* **1**(1), 3–30 (1998)
41. Synopsys, I.: Synopsys vcs. <https://www.synopsys.com/verification/simulation/vcs.html>
42. Synopsys, I.: Synopsys verdi. <https://www.synopsys.com/verification/debug/verdi.html>
43. Van Bulck, J., Weisse, K., Genkin, D., Minkin, M., Kasikci, B., Piessens, F., Silberstein, M., Fogh, A., Eisenbarth, T., Strackx, R.: Foreshadow: Extracting the keys to the intel sgx kingdom with transient out-of-order execution. *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)* pp. 991–1008 (2018)
44. Van Schaik, S., Milburn, M., Ragab, M., Bhasin, S., Borrello, M., Bos, H., Giuffrida, C., Razavi, K.: Rogue in-flight data load. *arXiv preprint arXiv:1905.01989* (2019)
45. Wang, P.H., Wang, H., Collins, J.D., Grochowski, E., Kling, R.M., Shen, J.P.: Memory latency-tolerance approaches for itanium processors: out-of-order execution vs. speculative precomputation. In: *Proceedings Eighth International Symposium on High Performance Computer Architecture*. pp. 187–196. IEEE (2002)
46. Weisse, O., Neal, I., Loughlin, K., Wenisch, T.F., Kasikci, B.: Nda: Preventing speculative execution attacks at their source. In: *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. pp. 572–586 (2019)
47. Wu, Y., Qian, X.: Reversispec: Reversible coherence protocol for defending transient attacks. *arXiv preprint arXiv:2006.16535* (2020)
48. XFusion Software LLC, I.: Timegen. https://www.xfusionsoftware.com/download_timegen.html
49. Xiong, W., Szefer, J.: Survey of transient execution attacks and their mitigations. *ACM Comput. Surv.* **54**(3) (May 2021). <https://doi.org/10.1145/3442479>, <https://doi.org/10.1145/3442479>
50. Yan, M., Choi, J., Skarlatos, D., Morrison, A., Fletcher, C., Torrellas, J.: Invisispec: Making speculative execution invisible in the cache hierarchy. In: *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. pp. 428–441. IEEE (2018)

51. Yan, M., Sprabery, R., Gopireddy, B., Fletcher, C., Campbell, R., Torrellas, J.: Attack directories, not caches: Side channel attacks in a non-inclusive world. In: 2019 IEEE Symposium on Security and Privacy (SP). pp. 888–904. IEEE (2019)
52. Yarom, Y., Falkner, K.: {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In: 23rd USENIX security symposium (USENIX security 14). pp. 719–732 (2014)
53. Yu, J., Yan, M., Khyzha, A., Morrison, A., Torrellas, J., Fletcher, C.W.: Speculative taint tracking (stt) a comprehensive protection for speculatively accessed data. In: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture. pp. 954–968 (2019)
54. Zhao, J., Korpan, B., Gonzalez, A., Asanovic, K.: Sonicboom: The 3rd generation berkeley out-of-order machine. In: Fourth Workshop on Computer Architecture Research with RISC-V. vol. 5, pp. 1–7 (2020)